

数值解析法 II

第10回 数值線形代数(2):前処理

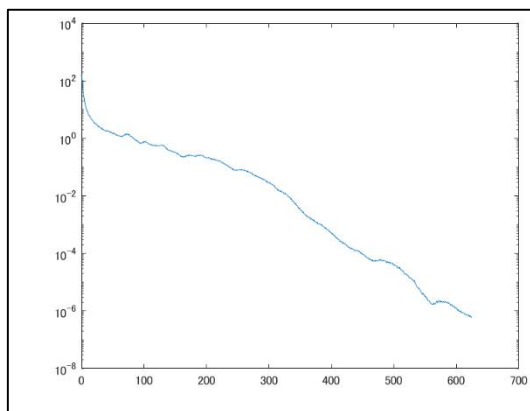
前処理

前回の例題

次の手順で正定値行列を作成し、共役勾配法の MATLAB プログラムを作成・実行する．ただし、真の解ベクトルは

$$\boldsymbol{x}^* = (1 \ 1 \ 1 \ \cdots \ 1)^T \in \mathbb{R}^n$$

とし、右辺ベクトルを $\boldsymbol{b} = A\boldsymbol{x}^*$ により生成する．また、 $\varepsilon = 1.0 \times 10^{-8}$ であり、反復回数が $2n$ を超えた場合も終了する

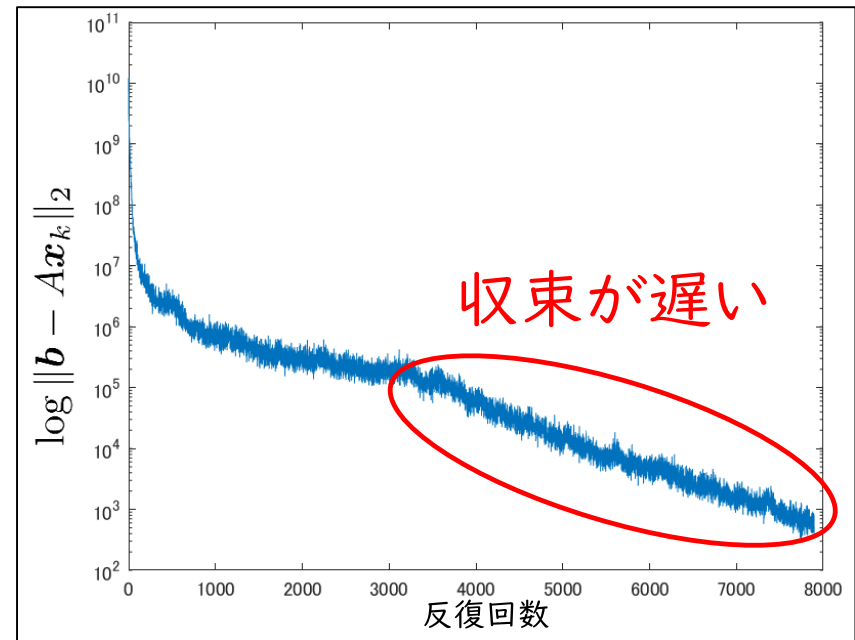
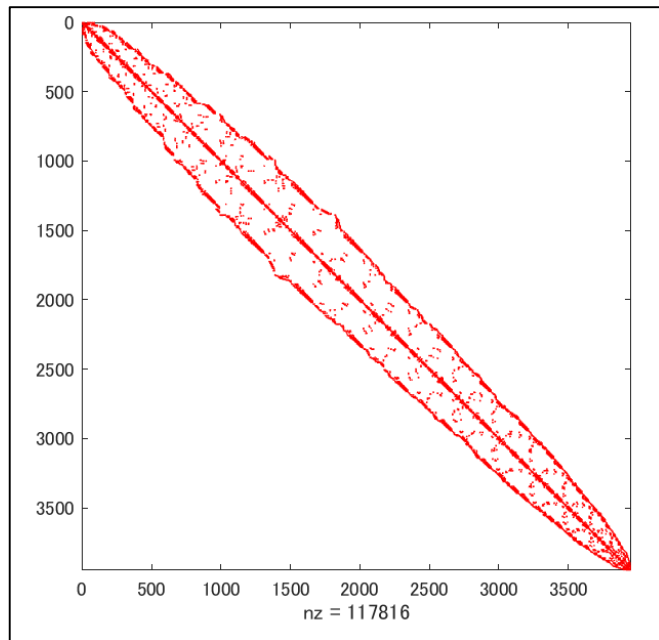


[Suite Sparse Matrix Collection:Kuu.dat](#)

反復回数：629 相対誤差 $\approx 5.2001 \times 10^{-8}$

計算結果 (n=3948, 初期ベクトル: 乱数ベクトル, `bcsstk15.mat`)

最大反復回数で収束しない



片対数グラフ(残差)

定理（共役勾配法の収束_[杉浦・室田]）

$A \in \mathbb{R}^{n \times n}$ を実正定値行列とする.

- (1) 共役勾配法による反復列は高々 n ステップで $Ax = b$ の解へ収束する.
- (2) $\lambda_1 \geq \dots \geq \lambda_n > 0$ を A の固有値とし, $\kappa = \lambda_1 / \lambda_n$ とする.

$$\phi(\mathbf{x}_k) \leq 4 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^{2k} \phi(\mathbf{x}_0).$$

固有値が密集しているほどに収束が速くなる（前処理の動機）

今回の内容

前処理の概念

$$Ax = b$$

条件数 $\kappa(A) \geq 1$

1に近づければ
収束性アップ

前処理行列

正則行列 M

前処理

$$M^{-1}Ax = M^{-1}b$$

- 解 x は不変
- $\kappa(M^{-1}A)$ が $\kappa(A)$ に比べ
1 に近づければ CG 法の収束率アップ

一方, CG法適用にあたって...

- (i) $M^{-1}A$: 正定値行列の必要あり
- (ii) M^{-1} : 計算量の観点から好ましくない

前処理付き共役勾配法 (PCG法)

$$M := LL^T \quad (L : \text{正則な下三角行列})$$

$$M^{-1}Ax = M^{-1}b \quad \longrightarrow \quad L^{-T}L^{-1}Ax = L^{-T}L^{-1}b$$

$$\longrightarrow \quad L^{-T}L^{-1}AL^{-T}(L^Tx) = L^{-T}L^{-1}b$$

$$\longrightarrow \quad L^{-1}AL^{-T}(L^Tx) = L^{-1}b$$

A: 正定値行列ならば正定値行列

共役勾配法を適用可能

Algorithm 2 前処理付き共役勾配法

Require: 初期ベクトル $\mathbf{x}_0$;

$$\tilde{\mathbf{x}}_0 = L^T \mathbf{x}_0;$$

$$\tilde{\mathbf{r}}_0 = L^{-1} \mathbf{b} - L^{-1} A L^{-T} \tilde{\mathbf{x}}_0;$$

$$\tilde{\mathbf{p}}_0 = \tilde{\mathbf{r}}_0;$$

$$k = 0;$$

repeat

$$\alpha_k = \frac{(\tilde{\mathbf{r}}_k, \tilde{\mathbf{p}}_k)}{(\tilde{\mathbf{p}}_k, L^{-1} A L^{-T} \tilde{\mathbf{p}}_k)};$$

$$\tilde{\mathbf{x}}_{k+1} = \tilde{\mathbf{x}}_k + \alpha_k \tilde{\mathbf{p}}_k;$$

$$\tilde{\mathbf{r}}_{k+1} = \tilde{\mathbf{r}}_k - \alpha_k L^{-1} A L^{-T} \tilde{\mathbf{p}}_k;$$

$$\beta_k = -\frac{(\tilde{\mathbf{r}}_{k+1}, L^{-1} A L^{-T} \tilde{\mathbf{p}}_k)}{(\tilde{\mathbf{p}}_k, L^{-1} A L^{-T} \tilde{\mathbf{p}}_k)};$$

$$\tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{r}}_{k+1} + \beta_k \tilde{\mathbf{p}}_k,$$

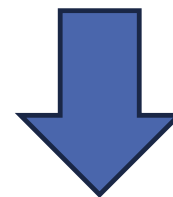
until $\|\tilde{\mathbf{r}}_{k+1}\|_2 < \varepsilon \|L^{-1} \mathbf{b}\|_2$

$$\mathbf{x}_{k+1} = L^{-T} \tilde{\mathbf{x}}_{k+1}$$

return $\mathbf{x}_{k+1}$;

- 表現が少し煩雑

- $\mathbf{x}_{k+1}$ を求めるのに逆行列が必要



簡約化された表現を検討

$$\tilde{\mathbf{x}}_k = L^T \mathbf{x}_k, \tilde{\mathbf{p}}_k = L^T \mathbf{p}_k$$

$$\bullet \tilde{\mathbf{r}}_k = L^{-1} \mathbf{b} - L^{-1} A L^{-T} \tilde{\mathbf{x}}_k = L^{-1} \mathbf{b} - L^{-1} A \mathbf{x}_k = L^{-1} \mathbf{r}_k$$

$$\bullet \alpha_k = \frac{(\tilde{\mathbf{r}}_k, \tilde{\mathbf{p}}_k)}{(\tilde{\mathbf{p}}_k, L^{-1} A L^{-T} \tilde{\mathbf{p}}_k)} = \frac{(L^{-1} \mathbf{r}_k, L^T \mathbf{p}_k)}{(L^T \mathbf{p}_k, L^{-1} A \mathbf{p}_k)} = \frac{(\mathbf{r}_k, \mathbf{p}_k)}{(\mathbf{p}_k, A \mathbf{p}_k)}$$

$$\bullet \tilde{\mathbf{x}}_{k+1} = \tilde{\mathbf{x}}_k + \alpha_k \tilde{\mathbf{p}}_k \Rightarrow L^T \mathbf{x}_{k+1} = L^T \mathbf{x}_k + \alpha_k L^T \mathbf{p}_k$$

$$\Rightarrow \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$$

$$\bullet \tilde{\mathbf{r}}_{k+1} = \tilde{\mathbf{r}}_k - \alpha_k L^{-1} A L^{-T} \tilde{\mathbf{p}}_k$$

$$\Rightarrow L^{-1} \mathbf{r}_{k+1} = L^{-1} \mathbf{r}_k - \alpha_k L^{-1} A \mathbf{p}_k$$

$$\Rightarrow \mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k$$

$$\tilde{\mathbf{x}}_k = L^T \mathbf{x}_k, \tilde{\mathbf{p}}_k = L^T \mathbf{p}_k$$

- $$\beta_k = -\frac{(\tilde{\mathbf{r}}_{k+1}, L^{-1} A L^{-T} \tilde{\mathbf{p}}_k)}{(\tilde{\mathbf{p}}_k, L^{-1} A L^{-T} \tilde{\mathbf{p}}_k)}$$

$$= \frac{(L^{-1} \mathbf{r}_{k+1}, L^{-1} A \mathbf{p}_k)}{(L^T \mathbf{p}_k, L^{-1} A \mathbf{p}_k)} = \frac{((LL^T)^{-1} \mathbf{r}_{k+1}, A \mathbf{p}_k)}{(\mathbf{p}_k, A \mathbf{p}_k)}$$
- $$\tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{r}}_{k+1} + \beta_k \tilde{\mathbf{p}}_k \Rightarrow L^T \mathbf{p}_{k+1} = L^{-1} \mathbf{r}_{k+1} + \beta_k L^T \mathbf{p}_k$$

$$\Rightarrow \mathbf{p}_{k+1} = (LL^T)^{-1} \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$$

これらのAlgorithm2に適用⇒前処理付き共役勾配法 (PCG法)

Algorithm 3 前処理付き共役勾配法 (PCG 法) : 簡略化版

Require: 初期ベクトル $\mathbf{x}_0$;

$$\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0;$$

$$\mathbf{p}_0 = (LL^T)^{-1}\mathbf{r}_0;$$

$$L\mathbf{u} = \mathbf{r} \implies L^T\mathbf{v} = \mathbf{u}$$

逆行列を計算する
必要はない

$$k = 0;$$

前進代入&後退代入でOK

repeat

$$\alpha_k = \frac{(\mathbf{r}_k, \mathbf{p}_k)}{(\mathbf{p}_k, A\mathbf{p}_k)};$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k;$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A\mathbf{p}_k;$$

$$\beta_k = -\frac{((LL^T)^{-1}\mathbf{r}_{k+1}, A\mathbf{p}_k)}{(\mathbf{p}_k, A\mathbf{p}_k)};$$

$$\mathbf{p}_{k+1} = (LL^T)^{-1}\mathbf{r}_{k+1} + \beta_k \mathbf{p}_k;$$

until $\|\mathbf{r}_{k+1}\|_2 < \varepsilon \|\mathbf{b}\|_2$

return $\mathbf{x}_{k+1}$;

これで問題ないが...

Algorithm 1 共役勾配法

Require: 初期ベクトル $\mathbf{x}_0$;

$$\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0;$$

$$\mathbf{p}_0 = \mathbf{r}_0;$$

$$k = 0;$$

repeat

$$\alpha_k = \frac{(\mathbf{r}_k, \mathbf{p}_k)}{(\mathbf{p}_k, A\mathbf{p}_k)}; \quad (\mathbf{r}_k, \mathbf{p}_k) = (\mathbf{r}_k, \mathbf{r}_k + \beta_{k-1}\mathbf{p}_{k-1}) = (\mathbf{r}_k, \mathbf{r}_k)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k;$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A\mathbf{p}_k;$$

$$\beta_k = -\frac{(\mathbf{r}_{k+1}, A\mathbf{p}_k)}{(\mathbf{p}_k, A\mathbf{p}_k)} = \left(\mathbf{r}_{k+1}, \frac{1}{\alpha_k}(\mathbf{r}_k - \mathbf{r}_{k+1}) \right) = -\frac{1}{\alpha_k}(\mathbf{r}_{k+1}, \mathbf{r}_{k+1})$$

$$= (\mathbf{r}_k + \beta_{k-1}\mathbf{p}_{k-1}, A\mathbf{p}_k)$$

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k;$$

$$\text{until } \|\mathbf{r}_{k+1}\|_2 < \varepsilon \|\mathbf{b}\|_2 \quad = (\mathbf{r}_k, A\mathbf{p}_k) = \frac{1}{\alpha_k}(\mathbf{r}_k, \mathbf{r}_k)$$

return $\mathbf{x}_{k+1}$;

- $(\mathbf{r}_k, \mathbf{p}_j) = 0 \quad (j \leq k-1)$
- $(\mathbf{r}_k, \mathbf{r}_j) = 0 \quad (j \leq k-1)$
 $(\because \mathbf{r}_j = \text{span}(\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_j))$

$$\beta_k = \frac{(\mathbf{r}_{k+1}, \mathbf{r}_{k+1})}{(\mathbf{r}_k, \mathbf{r}_k)}$$

Algorithm 3 前処理付き共役勾配法 (PCG 法)：簡略化版 2

Require: 初期ベクトル $\mathbf{x}_0$;

$$\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0;$$

$$\mathbf{p}_0 = (LL^T)^{-1}\mathbf{r}_0;$$

$$k = 0; \quad \text{各ベクトル列: 毎ステップ保存しておく必要なし}$$

repeat

$$\alpha_k = \frac{((LL^T)^{-1}\mathbf{r}_k, \mathbf{r}_k)}{(\mathbf{p}_k, A\mathbf{p}_k)};$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k; \quad \text{各ステップ1回計算して保存すれば 計算効率アップ}$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A\mathbf{p}_k;$$

$$\beta_k = \frac{((LL^T)^{-1}\mathbf{r}_{k+1}, \mathbf{r}_{k+1})}{((LL^T)^{-1}\mathbf{r}_k, \mathbf{r}_k)};$$

次のステップのために保存すれば
計算効率アップ

$$\mathbf{p}_{k+1} = (LL^T)^{-1}\mathbf{r}_{k+1} + \beta_k \mathbf{p}_k;$$

until $\|\mathbf{r}_{k+1}\|_2 < \varepsilon \|\mathbf{b}\|_2$

return $\mathbf{x}_{k+1}$;

Algorithm 4 前処理付き共役勾配法 (PCG 法) : 簡略化 + 省メモリ版

Require: 初期ベクトル $\mathbf{x}$;

$$\mathbf{r} = \mathbf{b} - A\mathbf{x};$$

$$\mathbf{p} = (LL^T)^{-1}\mathbf{r};$$

$$\rho = (\mathbf{p}, \mathbf{r});$$

repeat

$$\mathbf{q} = A\mathbf{p}; \gamma = (\mathbf{p}, \mathbf{q}); \alpha = \rho/\gamma;$$

$$\mathbf{x} = \mathbf{x} + \alpha\mathbf{p};$$

$$\mathbf{r} = \mathbf{r} - \alpha\mathbf{q};$$

$$\mathbf{q} = (LL^T)^{-1}\mathbf{r}; \mu = (\mathbf{q}, \mathbf{r});$$

$$\beta = \mu/\rho; \rho = \mu;$$

$$\mathbf{p} = \mathbf{q} + \beta\mathbf{p};$$

until $\|\mathbf{r}\|_2 < \varepsilon\|\mathbf{b}\|_2$

return $\mathbf{x}$;

前処理なしCG法プログラムを
微修正することで作成可能

代表的な前処理：不完全コレスキー分解

定理 (部分ピボット選択なしでLU分解可能な行列)

$A = (a_{ij})$ を n 次正方行列とする. A が次のいずれかの条件を満たすとき, 部分ピボット選択なしで LU 分解可能である.

(1) A が行方向に一般化狭義優対角, すなわち

$$\exists d_j \in \mathbb{R} \ (d_j > 0; 1 \leq j \leq n); \quad |a_{ii}| d_i > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| d_j \quad (1 \leq i \leq n).$$

(2) A^T が行方向に一般化狭義優対角である.

(3) A が正定値, すなわち $x^T A x > 0 \ (\forall x \neq 0)$.

$$A = S S^T \quad (S : \text{下三角行列})$$

コレスキー分解

【復習：コレスキー分解】

LU分解の
行列U

$$U = \begin{pmatrix} a_{11}^{(0)} & a_{12}^{(0)} & \cdots & a_{1n}^{(0)} \\ 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn}^{(n-1)} \end{pmatrix}$$

正定値性より

$$a_{kk}^{(k-1)} > 0$$

 D とおく

$$= \begin{pmatrix} a_{11}^{(0)} & 0 & \cdots & 0 \\ 0 & a_{22}^{(1)} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn}^{(n-1)} \end{pmatrix}$$

対称性より $= L^T$

$$\begin{pmatrix} 1 & a_{12}^{(0)} / a_{11}^{(0)} & \cdots & a_{1n}^{(0)} / a_{11}^{(0)} \\ 0 & 1 & \cdots & a_{2n}^{(1)} / a_{22}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix}$$



$$A = LDL^T$$

(A の LDL^T 分解)

$$D^{\frac{1}{2}} = \begin{pmatrix} \sqrt{a_{11}^{(0)}} & 0 & \cdots & 0 \\ 0 & \sqrt{a_{22}^{(1)}} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sqrt{a_{nn}^{(n-1)}} \end{pmatrix}, \quad S = LD^{\frac{1}{2}}$$

$$A = LD L^T$$



$$A = LD L^T = \left(LD^{\frac{1}{2}} \right) \left(LD^{\frac{1}{2}} \right)^T = S S^T$$

A のコレスキー分解

$$A = SS^T = \begin{pmatrix} \mathbf{s}_1 & \mathbf{s}_2 & \cdots & \mathbf{s}_n \end{pmatrix} \begin{pmatrix} \mathbf{s}_1^T \\ \mathbf{s}_2^T \\ \vdots \\ \mathbf{s}_n^T \end{pmatrix}$$

$$\mathbf{s}_k = (0, \cdots, 0, s_{kk}, \cdots, s_{nk})^T$$

$$A = SS^T = \begin{pmatrix} \mathbf{s}_1 & \mathbf{s}_2 & \cdots & \mathbf{s}_n \end{pmatrix} \begin{pmatrix} \mathbf{s}_1^T \\ \mathbf{s}_2^T \\ \vdots \\ \mathbf{s}_n^T \end{pmatrix}$$

$$\mathbf{s}_k = (0, \cdots, 0, s_{kk}, \cdots, s_{nk})^T$$

$$A^{(k)} = \begin{pmatrix} O_{k-1, k-1} & O_{k-1, n-k+1} \\ O_{n-k+1, k-1} & (a_{ij}^{(k)})_{k \leq i, j \leq n} \end{pmatrix}$$

➡ $A^{(k)} = \mathbf{s}_k \mathbf{s}_k^T + A^{(k+1)}, A^{(1)} = A$

$k = 1$

➡
$$\begin{aligned} s_{11} &= \sqrt{a_{11}^{(1)}}, s_{i1} = a_{i1}^{(1)} / s_{11} \quad (2 \leq j \leq n) \\ a_{ij}^{(2)} &= a_{ij}^{(1)} - s_{i1} s_{j1} \quad (2 \leq i, j \leq n) \end{aligned}$$

以下同様に
繰り返すことで
分解可能

Algorithm 5 コレスキー分解

Require: 正定値行列 $A = (a_{ij})$;

$$s_{11} = \sqrt{a_{11}};$$

for $i = 2$ to $n - 1$ **do****for all** $j = 1$ to $i - 1$ **do****for all** $k = 2$ to $j - 1$ **do**

$$s_{ij} = s_{ij} + s_{ik}s_{jk};$$

end for

$$s_{ij} = (a_{ij} - s_{ij})/s_{jj};$$

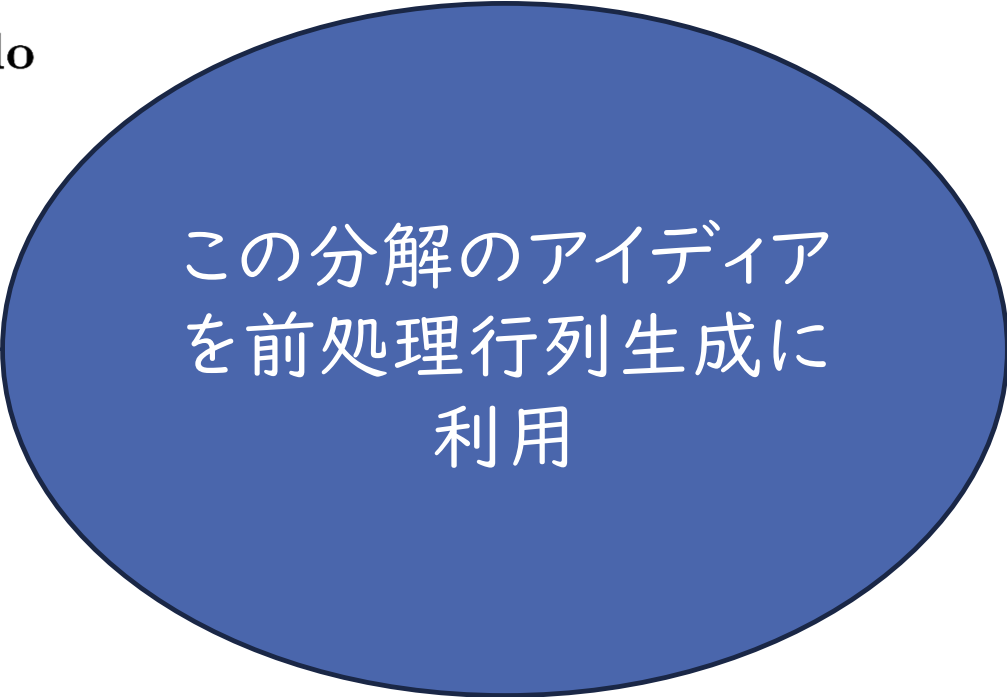
end for**for all** $k = 1$ to $i - 1$ **do**

$$s_{ii} = s_{ii} + s_{ik}^2;$$

end for

$$s_{ii} = \sqrt{a_{ii} - s_{ii}};$$

end for



この分解のアイデア
を前処理行列生成に
利用

Algorithm 4 前処理付き共役勾配法 (PCG 法) : 簡略化 + 省メモリ版**Require:** 初期ベクトル x ;

$$r = b - Ax;$$

$$p = (LL^T)^{-1}r;$$

$$\rho = (p, r);$$

repeat

$$q = Ap; \gamma = (p, q); \alpha = \rho/\gamma;$$

$$x = x + \alpha p;$$

$$r = r - \alpha q;$$

$$q = (LL^T)^{-1}r; \mu = (q, r);$$

$$\beta = \mu/\rho; \rho = \mu;$$

$$p = q + \beta p;$$

until $\|r\|_2 < \varepsilon \|b\|_2$ **return** x ;

$$Lu = r \implies L^T v = u$$

連立一次方程式を
前進・後退代入で解く



L がスパース行列ならば
高速計算可能

+

$(LL^T)^{-1}$ が A の逆行列に
近ければ収束性アップ



不完全コレスキー分解

前処理行列 $L = (\ell_{ij})$

係数行列 A の構造インデックス集合

$$\leftarrow K = \{(i, j) \mid i > j \text{ かつ } \ell_{ij} = 0\}$$

注) A : スパース行列 $\Rightarrow$ 三角部分の構造を利用

↓
コレスキー分解のアイディア

$$A^{(k)} \approx \ell_k \ell_k^T + A^{(k+1)}$$

$$\ell_{kk} = \sqrt{a_{kk}^{(k)}}, \quad \ell_{ik} = \begin{cases} a_{ik}^{(k)} / \ell_{kk} & ((i, k) \notin K, i > k) \\ 0 & ((i, k) \in K, i > k) \end{cases}$$

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - \ell_{ik} \ell_{jk} \quad (k+1 \leq i, j \leq n)$$

不完全コレスキー分解: この手順で $L = (\ell_1 \ \ell_2 \ \cdots \ \ell_n)$ を求める

不完全コレスキー分解に対する注意事項

- 前ページの不完全コレスキー分解：**ichol(0)**と呼ばれる
- インデックス集合ではなく、**しきい値**を用いる方法も存在：

$$\ell_{ik} = \begin{cases} a_{ik}^{(k)} / \ell_{kk} & (|a_{ik}^{(k)} / \ell_{kk}| > \text{しきい値}, i > k) \\ 0 & ((|a_{ik}^{(k)} / \ell_{kk}| \leq \text{しきい値}, i > k) \end{cases}$$

- **正定値であっても不完全コレスキー分解ができない場合もある**
- 不完全コレスキー分解可能な行列例：**対称な一般化狭義優対角行列**
- MATLAB：**ichol関数**が存在：
 - ✓ **$L = \text{ichol}(A, \text{オプション})$**
 - ✓ 入力引数：行列A（スパースの必要有）
 - ✓ オプション：方法, しきい値などを指定（指定方法はスクリプト参照）

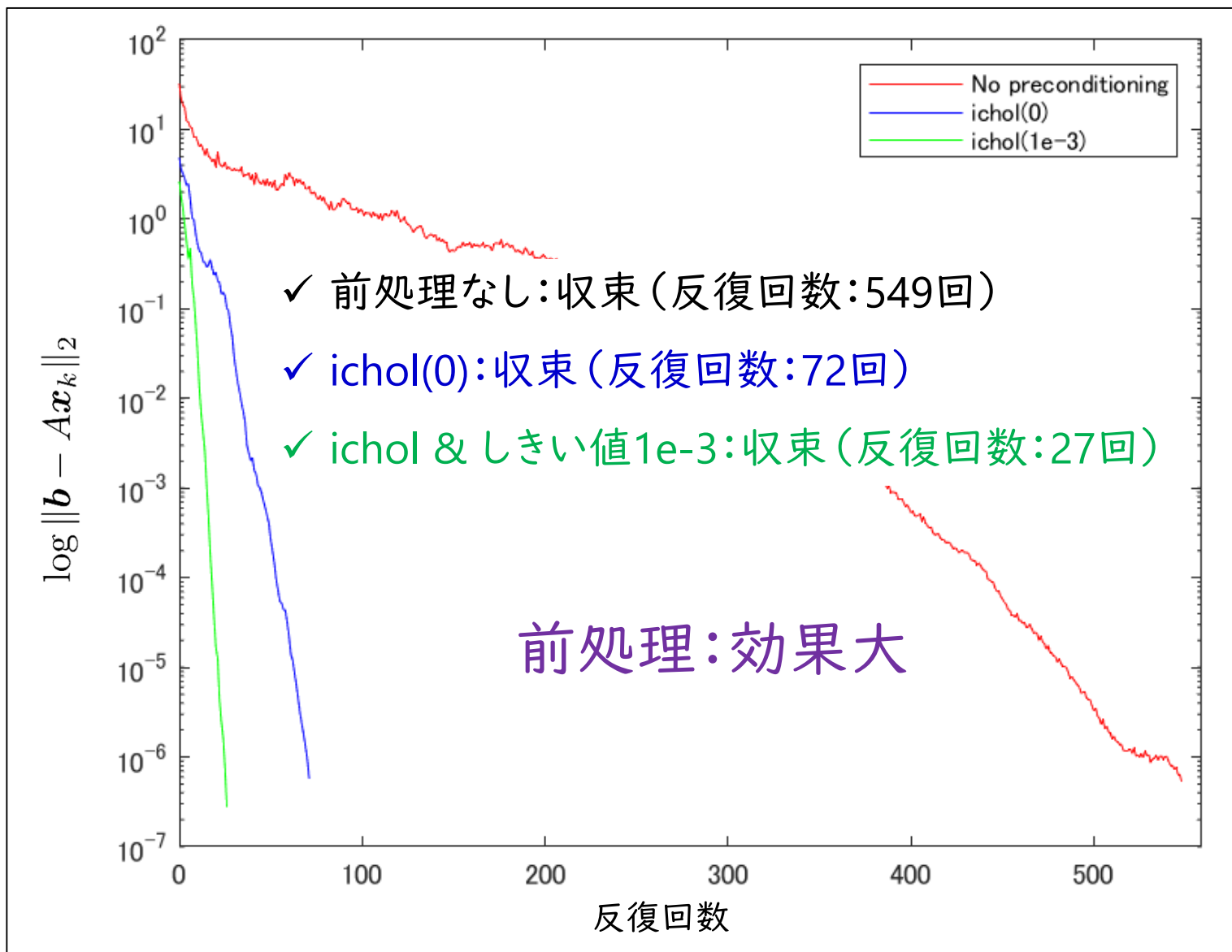
例題

前処理付き共役勾配法の MATLAB プログラムを作成・実行する．ただし，係数行列は前回講義で使ったもの（Kuu.mat, bcsstk15.mat）とし，真の解ベクトルは

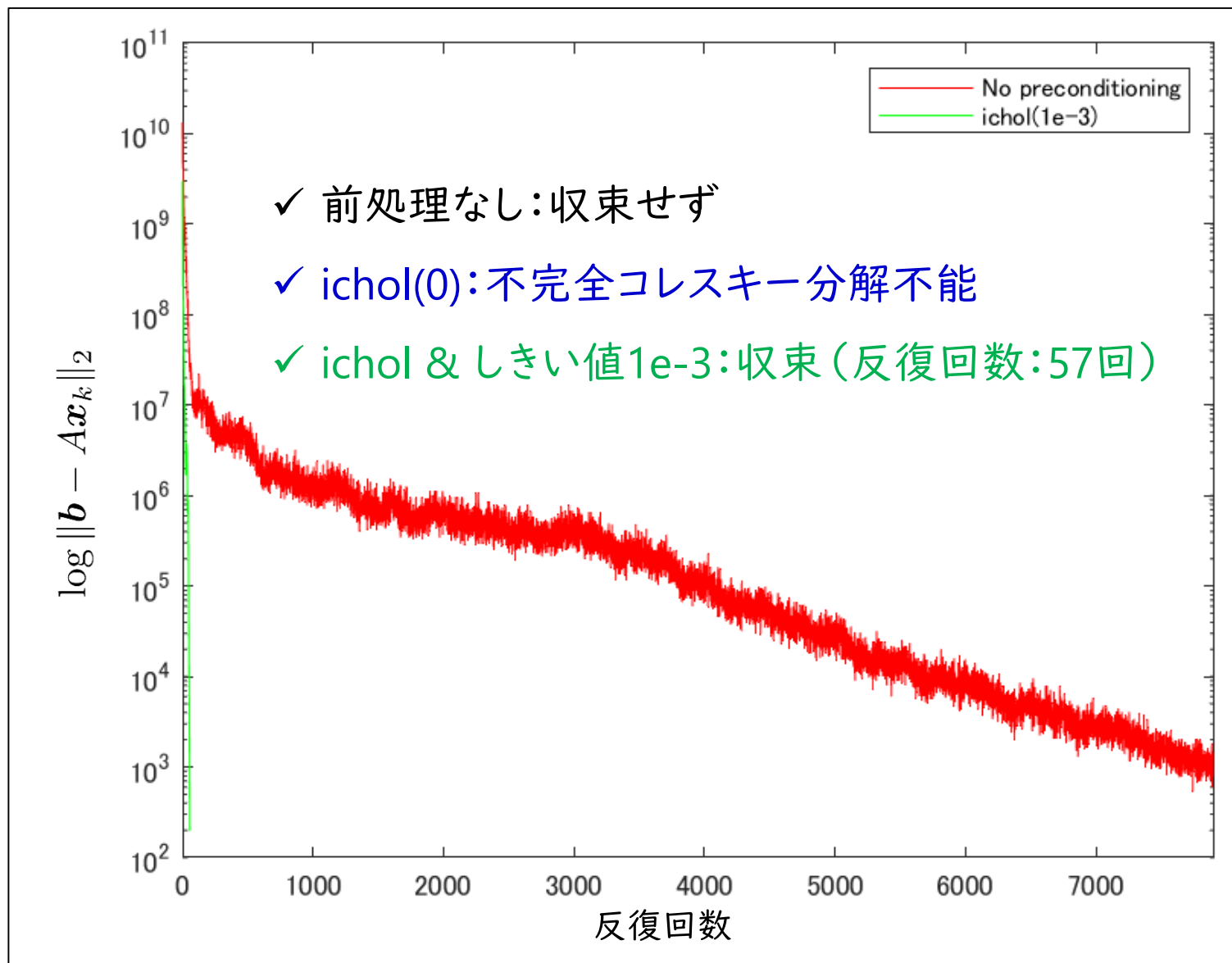
$$x^* = (1 \ 1 \ 1 \ \cdots \ 1)^T \in \mathbb{R}^n$$

とし，右辺ベクトルを $b = Ax^*$ により生成する．また，不完全コレスキー分解は `ichol` 関数を使用するものとし， $\varepsilon = 1.0 \times 10^{-8}$ ，反復回数が $2n$ を超えた場合も終了する

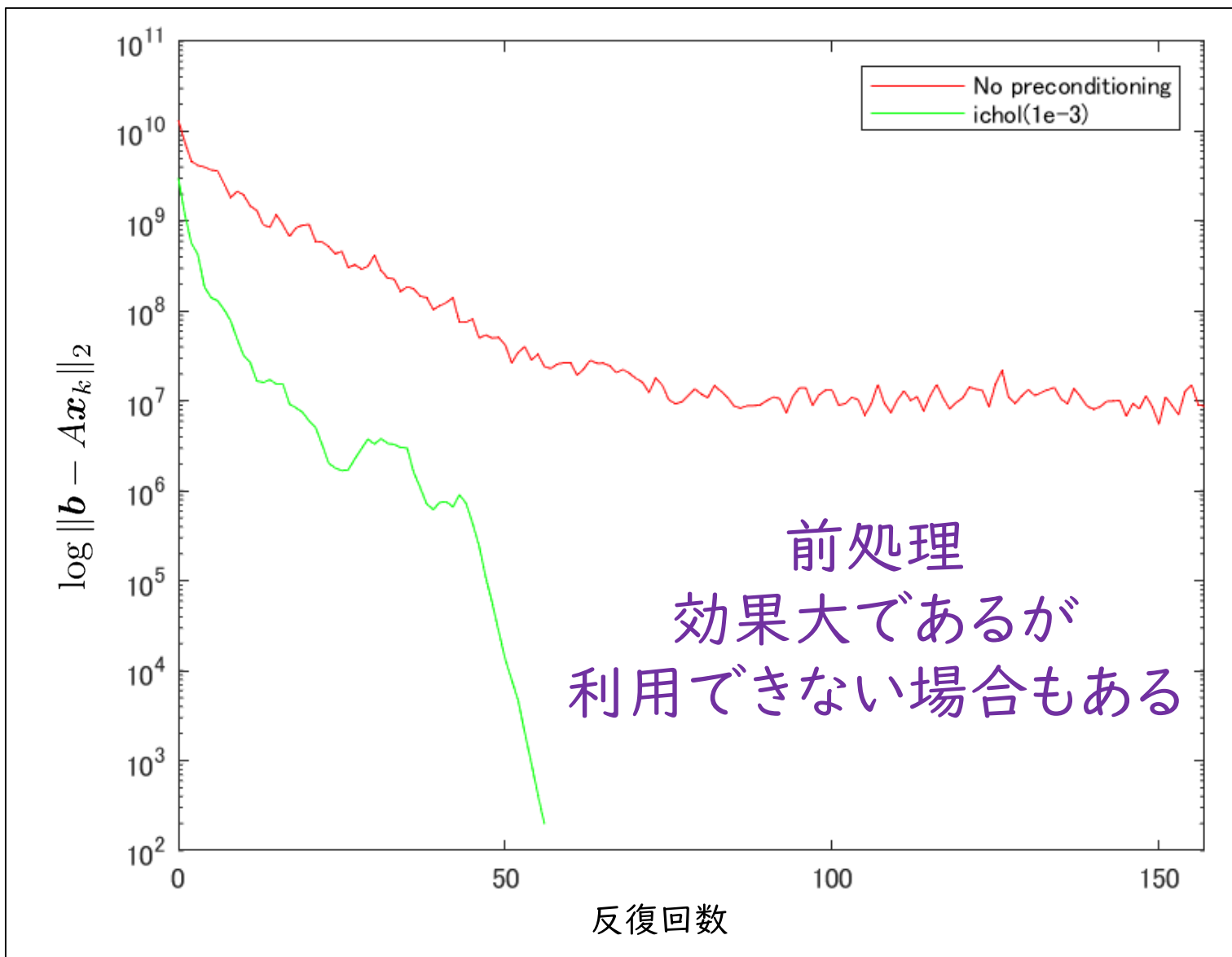
計算結果 (n=7102, 初期ベクトル: ゼロベクトル, Kuu.mat)



計算結果 (n=3948, 初期ベクトル: 乱数ベクトル, bcsstk15.mat)



計算結果 (n=3948, 初期ベクトル: 乱数ベクトル, bcsstk15.mat)



条件数 κ (Kuu.mat)

前処理なし	ichol(0)	ichol&しきい値
3.2553×10^4	1.6864×10^3	4.8123×10^2

条件数 κ (bcsstk15.mat)

前処理なし	ichol(0)	ichol&しきい値
7.9661×10^9	-	4.0788×10^4

条件数の改善が収束性に影響

MATLAB関数：pcgについて

使用例：スクリプト参照

➤ pcg関数

- ✓ $[x, \text{flag}, \text{relres}, \text{iter}, \text{resvec}] = \text{pcg}(A, b, \text{tol}, \text{maxit}, L)$
- ✓ 入力引数
 - A:行列, b:右辺ベクトル
 - tol:収束判定定数(省略した場合は $1e-6$)
 - maxit:最大反復回数(省略した場合は $\min(A\text{のサイズ}, 20)$)
 - L:前処理行列(省略した場合は前処理なし)
- ✓ 出力引数
 - ✓ x:解ベクトル
 - ✓ flag:収束フラグ(収束した場合は0, それ以外は失敗)
 - ✓ relres:相対残差誤差($\text{norm}(b - A*x) / \text{norm}(b)$)
 - ✓ iter:反復回数. resvec:残差ベクトル

演習

■ 問題3.4

- $A \in \mathbb{R}^{n \times n}$ を正定値行列, $L \in \mathbb{R}^{n \times n}$ を正則行列とする.
このとき, $L^{-1}AL^{-T}$ は正定値行列となることを示せ.

注) スライドでは L を正則な下三角行列としたが, 仮定としては正則性のみでよい

■ 問題3.5

- 前処理付き共役勾配法MATLABプログラムを作成し,
22ページの例題を計算しなさい.

注) ichol関数は使用可です.

まとめ

■ 今回の内容

□ 前処理

- ◆ 前処理：条件数を改善する行列を両辺にかける
- ◆ 前処理行列：正則な下三角行列を利用（正定値性を保持）
- ◆ 代表的な前処理：不完全コレスキー分解
- ◆ 適切な前処理行列：収束性を大きく改善
- ◆ 不完全コレスキー分解：必ず分解できるわけではない

■ 次回の内容

□ 残差直交性とLanczos原理